# Laporan Praktikum Modul 4
# Form, Interaksi, dan Integrasi Pustaka




**Disusun oleh:**

Salsabila Rahmadina
707012400067
D4SIKC-48-01
Pemprograman Perangkat Bergerak Lanjut

**Fakultas Ilmu Terapan**
**Telkom University**
**2026**

# A. Tujuan Praktikum

Praktikum ini bertujuan untuk memahami penggunaan Flutter dalam pengembangan aplikasi mobile yang melibatkan form, interaksi pengguna, dan pengelolaan state. Praktikum ini juga bertujuan untuk mempelajari penggunaan berbagai widget input dan interaksi, penerapan *Provider* dan *ChangeNotifier* untuk mengelola data, serta integrasi pustaka sesuai kebutuhan aplikasi. Melalui praktikum ini, mahasiswa diharapkan mampu menerapkan materi tersebut dalam membangun aplikasi mobile yang interaktif dan terstruktur.

# B. Alat dan Bahan

### Perangkat Keras (*Hardware*)

- Laptop/PC
- Prosesor: Intel Core i3 atau setara
- RAM: minimal 8 GB
- Penyimpanan: minimal 16 GB ruang kosong
- Layar: resolusi minimal 1366 × 768 piksel
- Koneksi internet, digunakan untuk instalasi paket dan mengunduh dependensi Flutter

### Perangkat Lunak (*Software*)

- Flutter SDK pada kanal stabil dan Dart SDK yang dikenali oleh perintah flutter doctor.
- Android Studio atau Visual Studio Code beserta ekstensi Flutter dan Dart.
- Paket *provider*. Tambahkan melalui perintah flutter pub add *provider* untuk melanjutkan pola pengelolaan state *ChangeNotifier* dan *Provider* dari praktikum sebelumnya.
- Paket *google_maps_flutter*. Tambahkan melalui perintah flutter pub add *google_maps_flutter* agar versi yang sesuai dicatat otomatis pada pubspec.yaml.
- Paket *fl_chart.* Tambahkan melalui perintah flutter pub add *fl_chart* untuk menampilkan data sebagai *chart* batang, garis, atau lingkaran.
- Paket ikon tambahan (opsional) seperti *flutter_chat_ui* atau *dash_chat_2* apabila ingin memakai komponen *chat* siap pakai, atau cukup gunakan *ListView* dan *TextField* bawaan Flutter untuk membangun tampilan *chat* sendiri.
- Kunci API Google Maps (Maps SDK for Android/iOS) yang telah diaktifkan pada Google Cloud Console, didaftarkan pada AndroidManifest.xml (Android) dan *AppDelegate*.swift (iOS).
- Git dan akun repositori untuk menyimpan hasil praktikum.

# C. Langkah-Langkah Praktikum

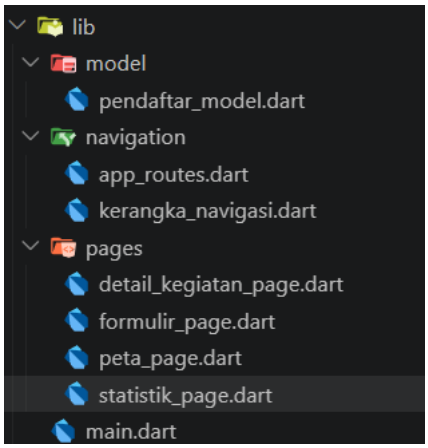
## 1. Membuat Proyek Praktikum

Praktikum diawali dengan membuat proyek Flutter baru bernama **form_pendaftaran_707012400067** melalui terminal Git Bash pada Visual Studio Code. Setelah proyek berhasil dibuat, terminal diarahkan ke folder proyek untuk menjalankan perintah berikutnya.

```
PS D:\Kuliah\Semester 5\Pemrograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_707012400067> flutter pub add provider
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ nested 1.0.0
+ provider 6.1.5+1
  test_api 0.7.12 (0.7.15 available)
  vector_math 2.4.0 (2.4.3 available)
  vm_service 15.3.0 (15.4.0 available)
Changed 2 dependencies!
Try `flutter pub outdated` for more information.
PS D:\Kuliah\Semester 5\Pemrograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_707012400067> flutter pub add google_maps_flutter
Resolving dependencies...
Downloading packages... (4.8s)
+ csslib 1.0.2
  cupertino_icons 1.0.9 (2.0.0 available)
+ flutter_plugin_android_lifecycle 2.0.35
+ flutter_web_plugins 0.0.0 from sdk flutter
+ google_maps 8.3.0
+ google_maps_flutter 2.19.0
+ google_maps_flutter_android 2.21.0
+ google_maps_flutter_ios 2.18.7
```

## 2. Menyiapkan Struktur Berkas Proyek

Struktur berkas disiapkan di dalam folder lib dengan membuat folder **model, pages, dan navigation**. Folder model digunakan untuk menyimpan model data pendaftar, folder pages digunakan untuk menyimpan halaman aplikasi, sedangkan folder navigation digunakan untuk mengatur navigasi dan route. Struktur ini membantu membuat kode program lebih terorganisir dan mudah dikembangkan.



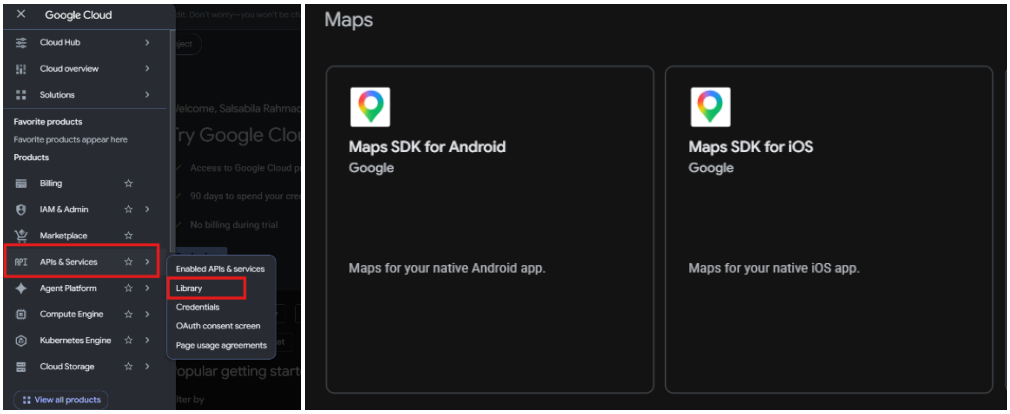

## 3. Konfigurasi Google Maps API Key pada Android

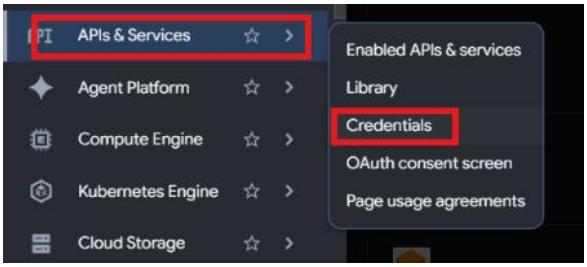
### a. Membuat dan Membatasi API Key

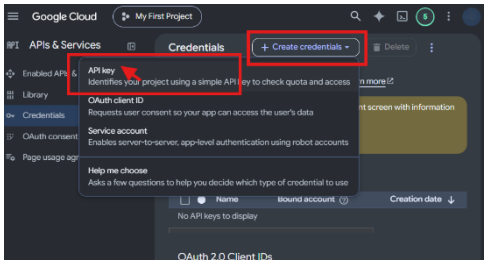
Setelah mengaktifkan **Maps SDK for Android** pada Google Cloud Console, langkah selanjutnya adalah membuat API Key untuk digunakan pada aplikasi Flutter. API Key dibatasi agar penggunaannya hanya diizinkan untuk aplikasi Android yang telah ditentukan.

**1. Membuat API Key**

Pada halaman Google Cloud Console (console.cloud.google.com), Pada *menu APIs & Services > Library,* aktifkan Maps SDK for Android dan Maps SDK for iOS, pilih menu *APIs & Services → Credentials*, kemudian pilih *Create Credentials → API Key*. Berikan nama API Key, misalnya PPBL Android.

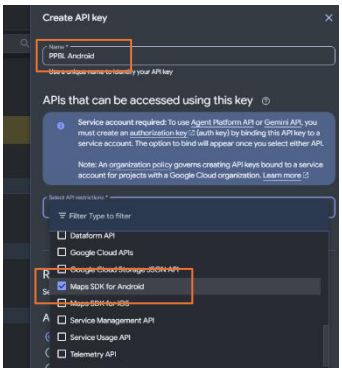


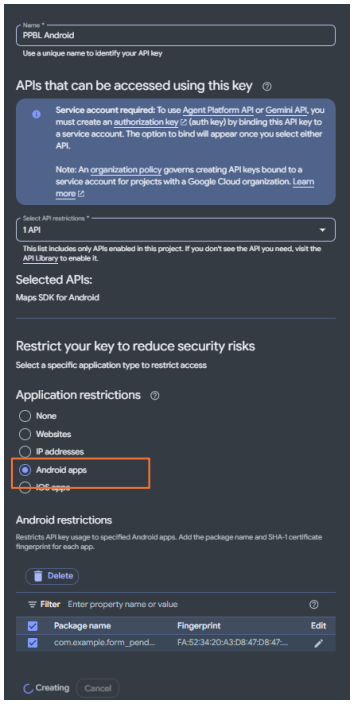

**2. Menentukan API yang Diizinkan**

Pada bagian *APIs that can be accessed using this key*, pilih **Maps SDK for Android**. Dengan pengaturan ini, API Key digunakan untuk mengakses layanan Google Maps pada aplikasi Android.




**3. Mengatur Application Restrictions**

Pada bagian *Application restrictions*, pilih **Android apps**. Pengaturan ini membatasi penggunaan API Key berdasarkan identitas aplikasi Android yang didaftarkan.

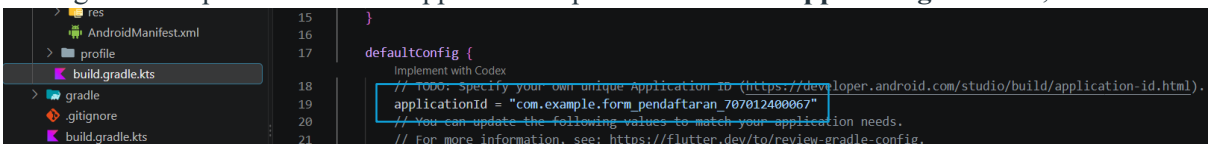



**4. Memasukkan Package Name dan SHA-1**

Klik tombol *Add* pada bagian Android restrictions, kemudian isi dua informasi berikut:

- **Package name:** com.example.form_pendaftaran_707012400067
- **SHA-1 certificate fingerprint:** sidik jari sertifikat yang diperoleh melalui perintah **.\gradlew signingReport** pada terminal di folder android.

Package name diperoleh dari nilai applicationId pada file **android/app/build.gradle.kts**,




sedangkan **SHA-1** diperoleh dari hasil perintah Gradle tersebut.

```
PS D:\Kuliah\Semester 5\Pemprograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4> cd form_pendaftaran_707012400067
PS D:\Kuliah\Semester 5\Pemprograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_707012400067> cd android
>> .\gradlew signingReport
```

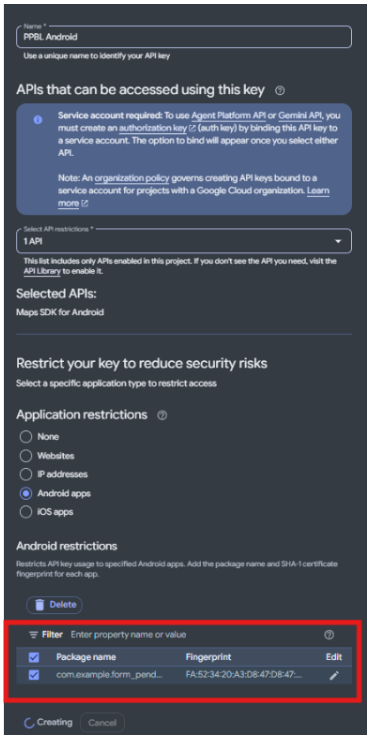
```
> Task :google_maps_flutter_android:signingReport
Variant: debugAndroidTest
Config: debug
Store: C:\Users\Sabil\.android\debug.keystore
Alias: AndroidDebugKey
MD5: 4F:FB:41:AF:2C:B9:2E:43:CA:84:1C:A2:DA:D0:6C:11
SHA1: FA:52:34:20:A3:D8:47:D8:47:A0:E6:4B:27:70:6C:21:C0:4D:8B:75
SHA-256: E3:2C:DD:D9:91:23:46:0F:C9:92:C0:51:BE:F7:02:DF:ED:92:C0:EA:03:56:A8:8D:65:04:CD:6E:F1:0D:AA:15
Valid until: Tuesday, 2 November 2055
----------

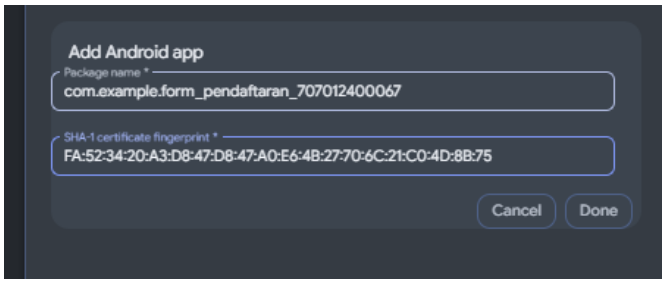
[Incubating] Problems report is available at: file:///D:/Kuliah/Semester%205/Pemprograman%20Perangkat%20Bergerak%20Lanjut/Praktikum/Pekan%204/Modul%
ports/problems/problems-report.html

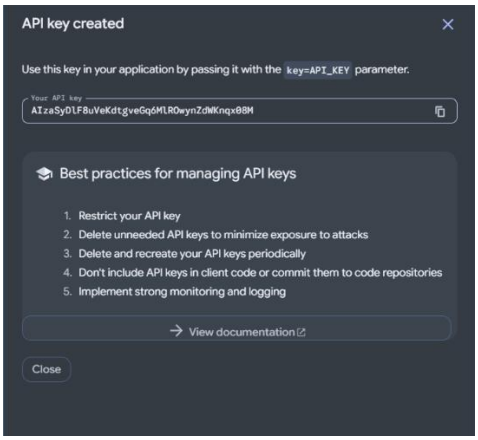
BUILD SUCCESSFUL in 1m 9s
7 actionable tasks: 4 executed, 3 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.1/userguide/configuration_cache_enabling.html
PS D:\Kuliah\Semester 5\Pemprograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_707012400067\android>
```

**5. Membuat API Key**

Pastikan nama aplikasi, package name, sidik jari SHA-1, dan API yang dipilih sudah sesuai. Selanjutnya, klik *Create* untuk membuat API Key Android yang telah dibatasi.

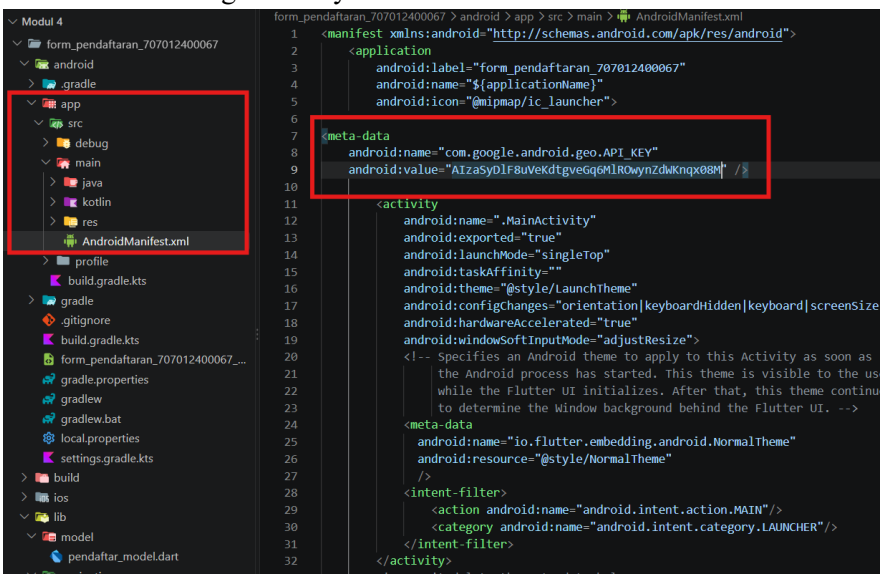



**b. Konfigurasi API Key pada AndroidManifest.xml**

Setelah API Key berhasil dibuat, kunci tersebut didaftarkan pada file **android/app/src/main/AndroidManifest.xml**. Penempatannya berada di dalam tag <application> dan sebelum tag <activity>.

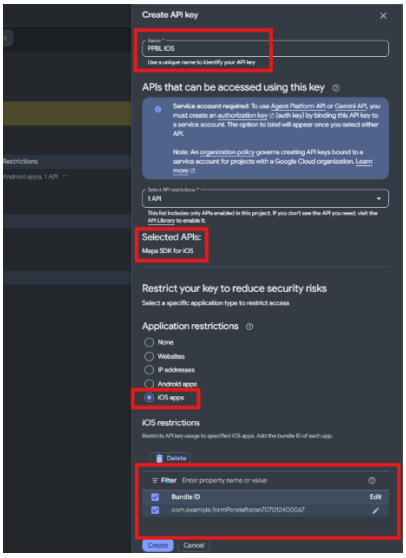


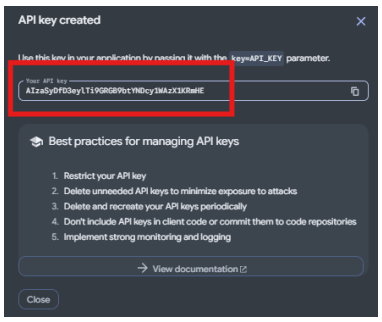

Ganti KUNCI_API_ANDROID_ANDA dengan API Key yang telah dibuat pada Google Cloud Console. Kode tersebut digunakan agar aplikasi Android dapat mengakses layanan Google Maps menggunakan API Key yang telah dikonfigurasi.

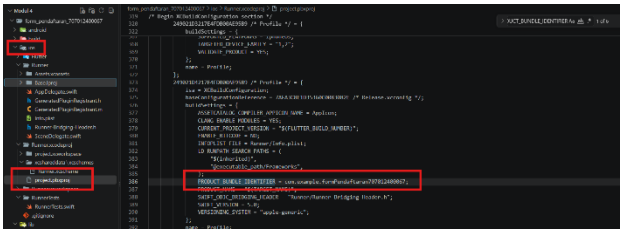
**4. Konfigurasi Google Maps API Key pada iOS**

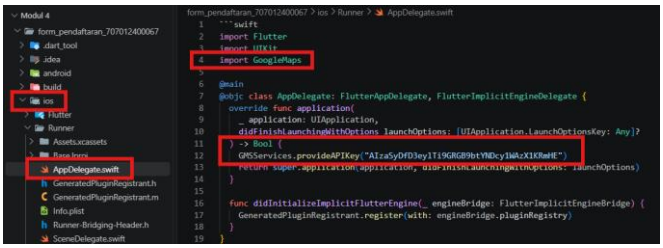
1. Membuat API Key untuk iOS melalui Google Cloud Console dengan memilih **Maps SDK for iOS**.
2. Mengatur pembatasan API Key dengan memilih **iOS apps**, kemudian memasukkan Bundle ID aplikasi, yaitu com.example.formPendaftaran707012400067.

3. Membuka file ios/Runner/AppDelegate.swift pada Visual Studio Code.
4. Menambahkan import GoogleMaps dan memasukkan API Key menggunakan GMSServices.provideAPIKey("KUNCI_API_IOS") di dalam metode application(_:didFinishLaunchingWithOptions:).






## 5. Hasil Konfigurasi iOS dan Error CocoaPods

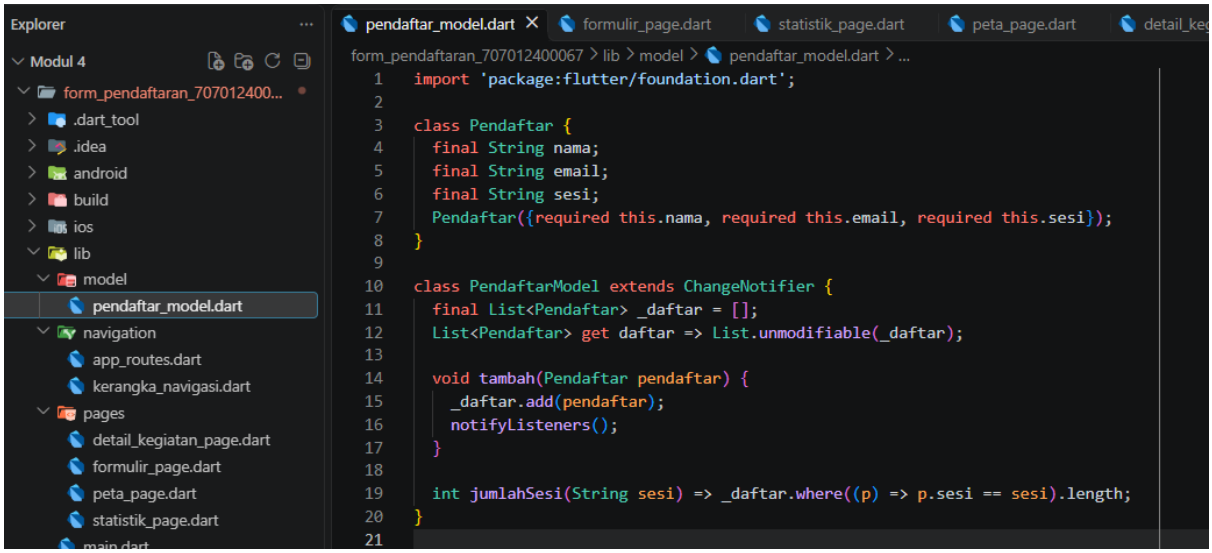
Pada tahap konfigurasi iOS, perintah cd ios berhasil dijalankan untuk masuk ke direktori ios pada project Flutter. Selanjutnya, perintah pod install dijalankan untuk memasang dependensi yang diperlukan.

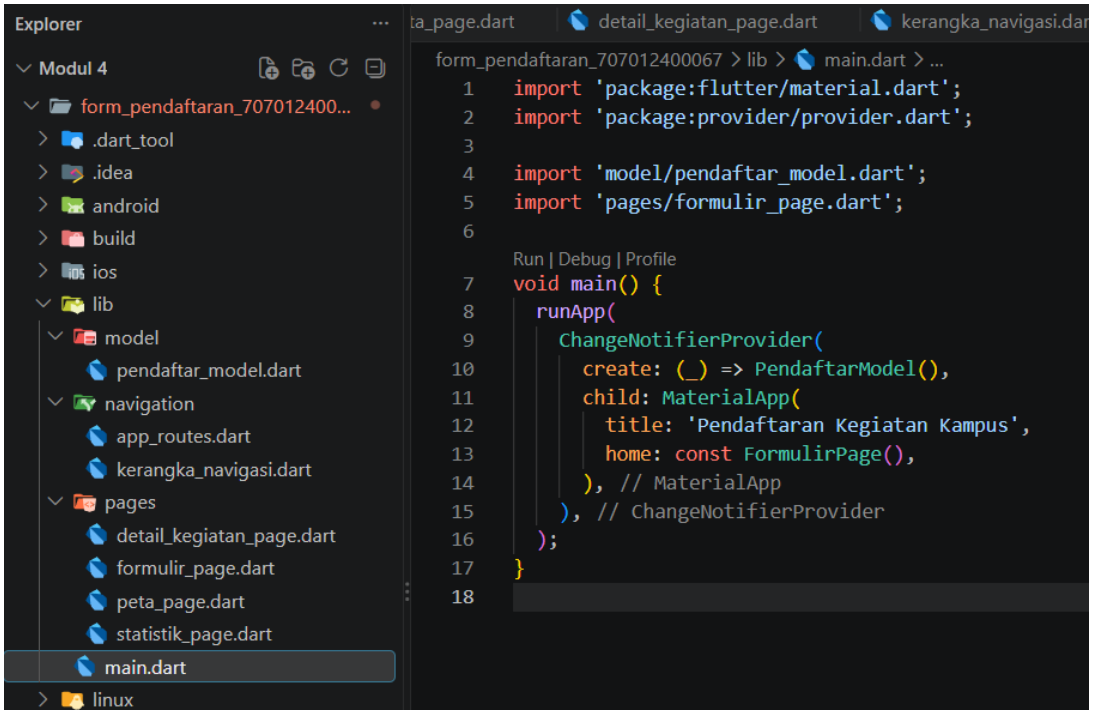
```
PS D:\Kuliah\Semester 5\Pemrograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_70701240006?> cd ios
PS D:\Kuliah\Semester 5\Pemrograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_70701240006?\ios> pod install
pod : The term 'pod' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
At line:1 char:1
+ pod install
+ ~~~
    + CategoryInfo          : ObjectNotFound: (pod:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

Error tersebut menunjukkan bahwa terminal PowerShell tidak mengenali perintah pod karena CocoaPods belum tersedia atau belum terkonfigurasi pada proyek yang digunakan.

## 6. Penyediaan PendaftarModel Menggunakan ChangeNotifierProvider

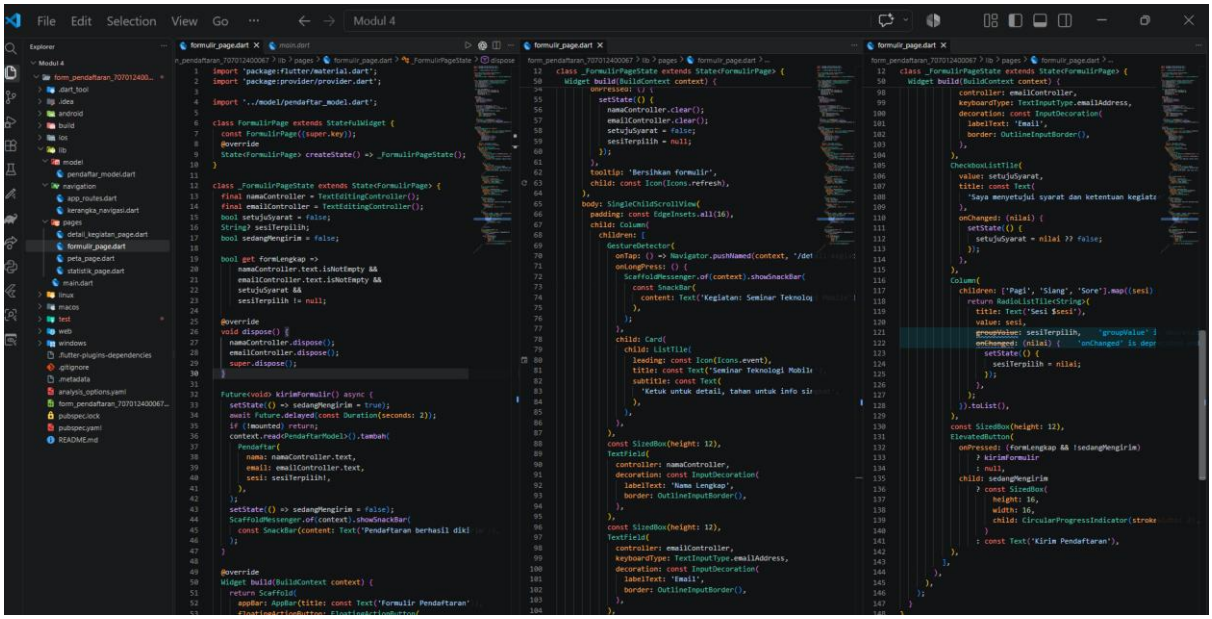
Pada tahap ini, dibuat file pendaftar_model.dart di dalam folder lib/model untuk mengelola data pendaftar kegiatan kampus. Model ini menggunakan ChangeNotifier agar perubahan data dapat diberitahukan kepada halaman yang membutuhkan.

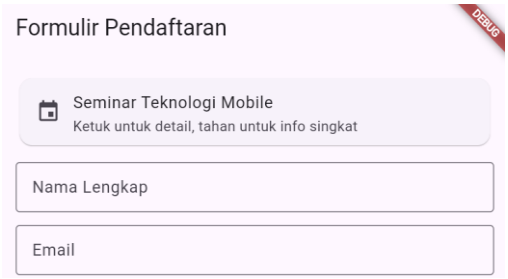



## 7. Membuat Form dengan TextField

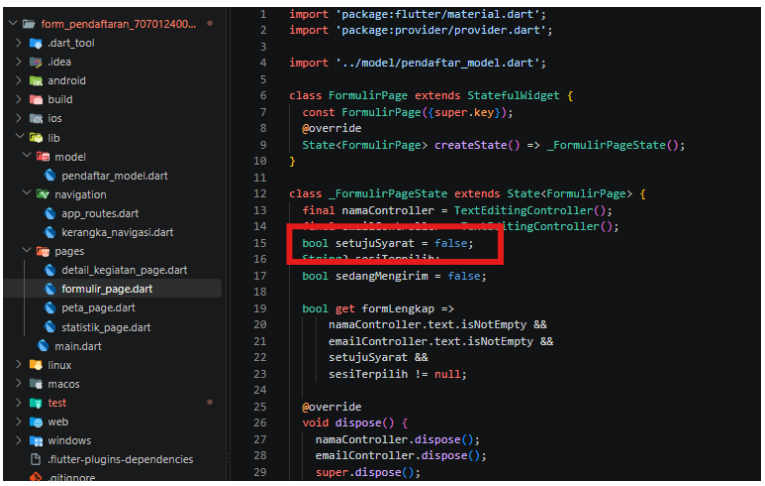
Pada tahap ini, dibuat file formulir_page.dart pada folder lib/pages sebagai halaman formulir pendaftaran. Halaman ini menggunakan StatefulWidget dan dua TextEditingController untuk menampung input nama dan email peserta.

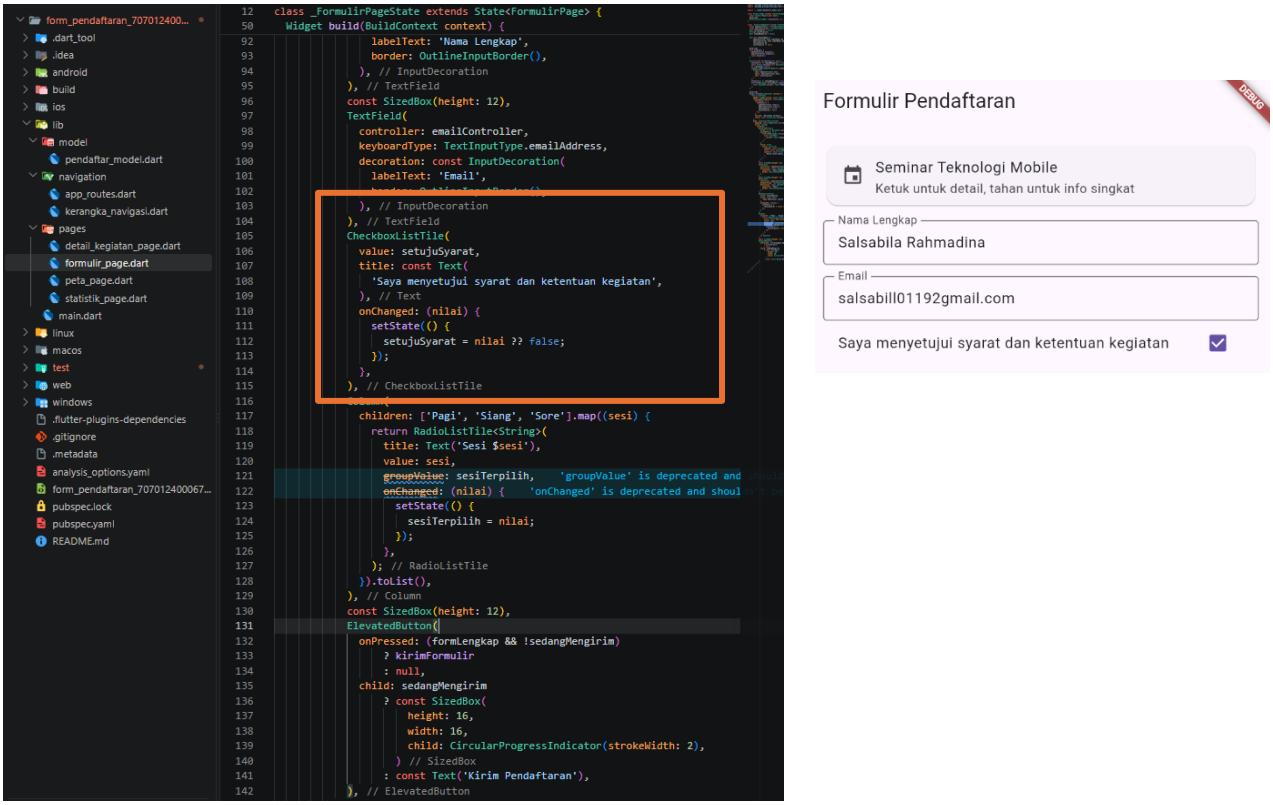





## 8. Menambahkan Checkbox

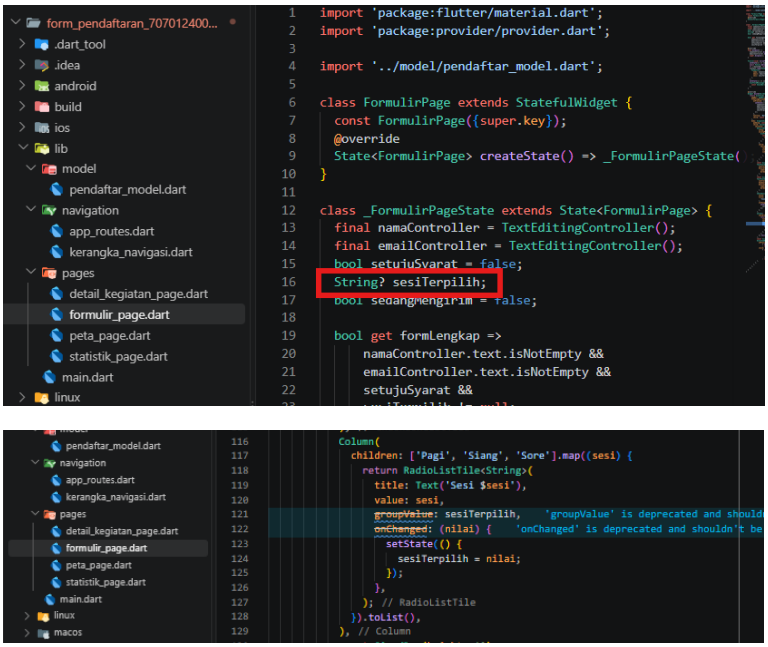
Pada tahap ini, ditambahkan komponen CheckboxListTile pada halaman formulir untuk menyetujui syarat dan ketentuan kegiatan. Variabel setujuSyarat digunakan untuk menyimpan status checkbox, sedangkan setState() memperbarui tampilan ketika checkbox dipilih atau dibatalkan.

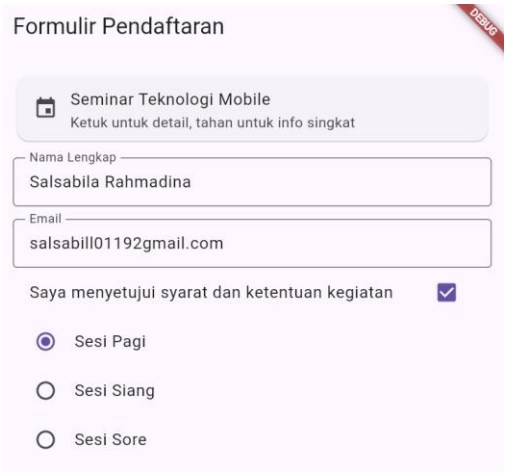





## 9. Menambahkan RadioButton

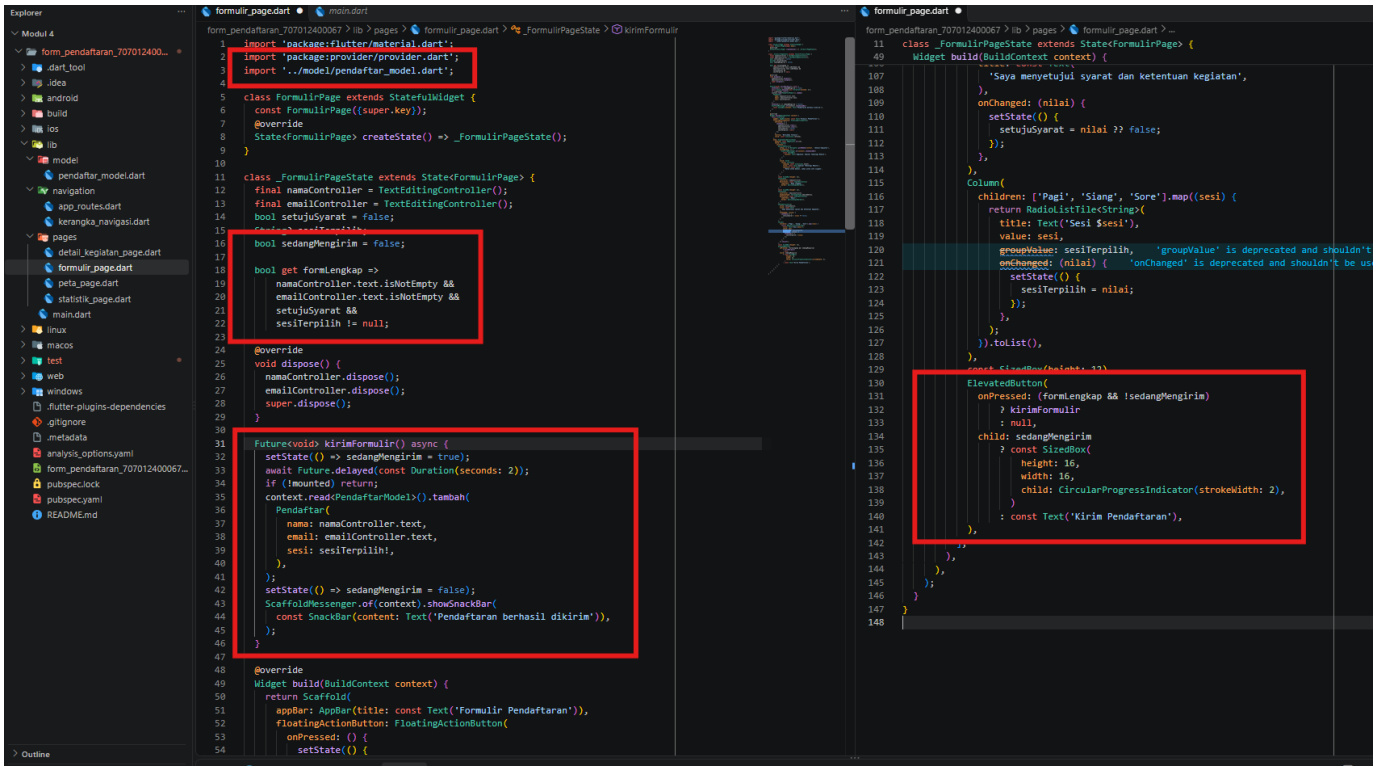
Pada tahap ini, ditambahkan pilihan sesi kegiatan menggunakan komponen RadioListTile, yaitu sesi Pagi, Siang, dan Sore. Variabel sesiTerpilih digunakan untuk menyimpan pilihan pengguna, sedangkan setState() memperbarui tampilan ketika pilihan sesi berubah.

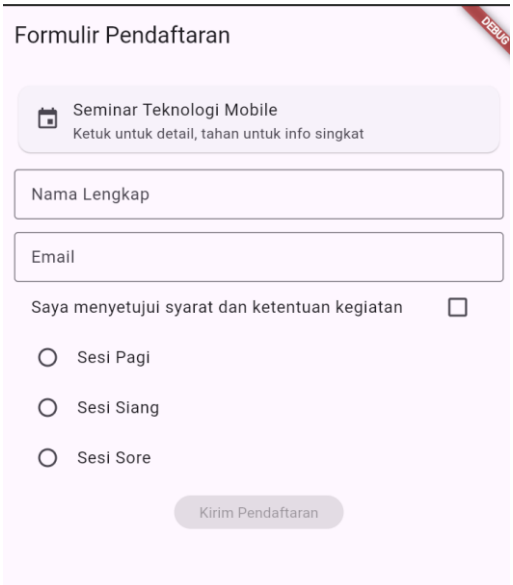





## 10. Menggunakan ElevatedButton untuk Mengirim Form

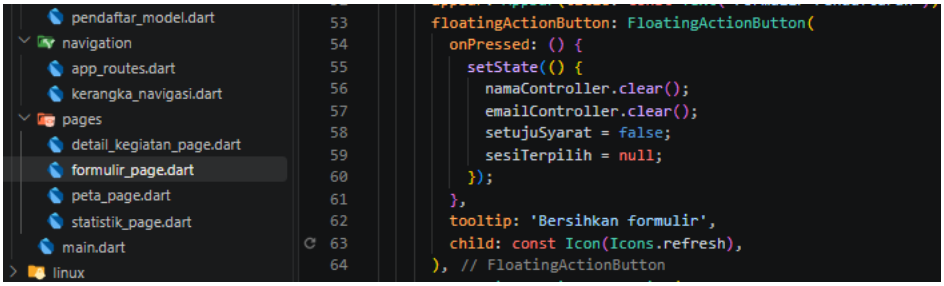
Pada tahap ini, ditambahkan tombol ElevatedButton untuk mengirim formulir pendaftaran. Tombol hanya aktif apabila seluruh data yang diperlukan telah diisi. Saat proses pengiriman berlangsung, tombol menampilkan indikator memuat. Data pendaftar kemudian disimpan ke PendaftarModel dan aplikasi menampilkan pesan bahwa pendaftaran berhasil dikirim.

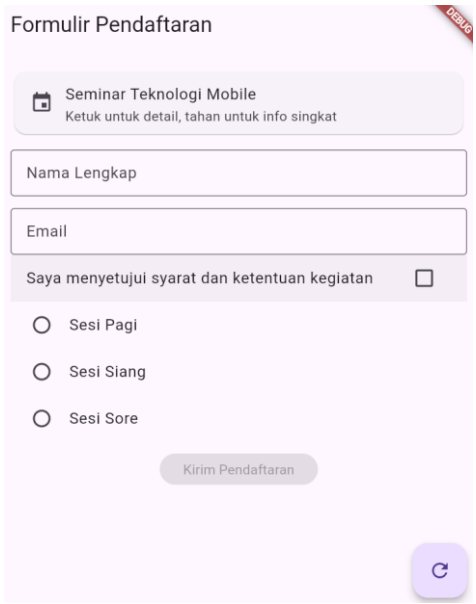





## 11. Menambahkan FloatingActionButton

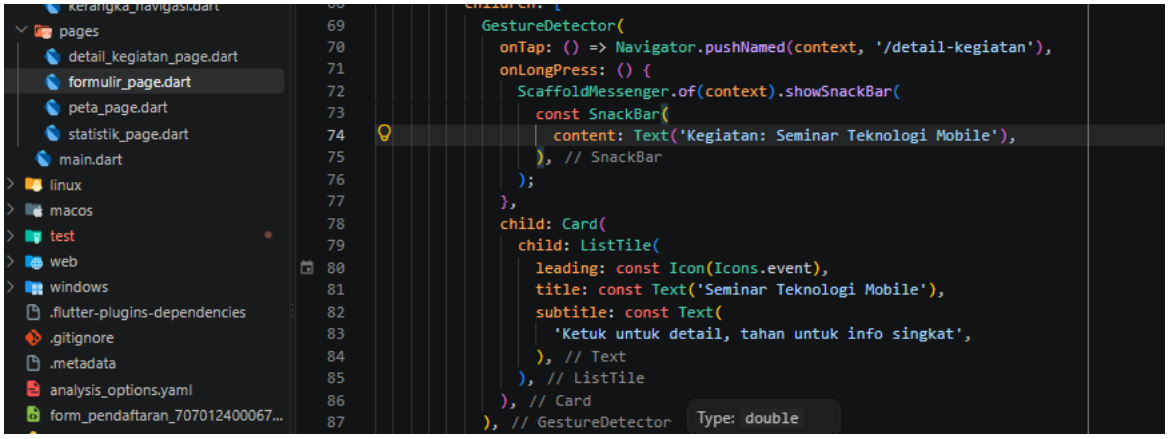
Pada tahap ini, ditambahkan FloatingActionButton dengan ikon refresh untuk mengosongkan formulir pendaftaran. Tombol menggunakan setState() untuk memperbarui tampilan, menghapus input nama dan email, serta mengembalikan status checkbox dan pilihan sesi ke kondisi awal.

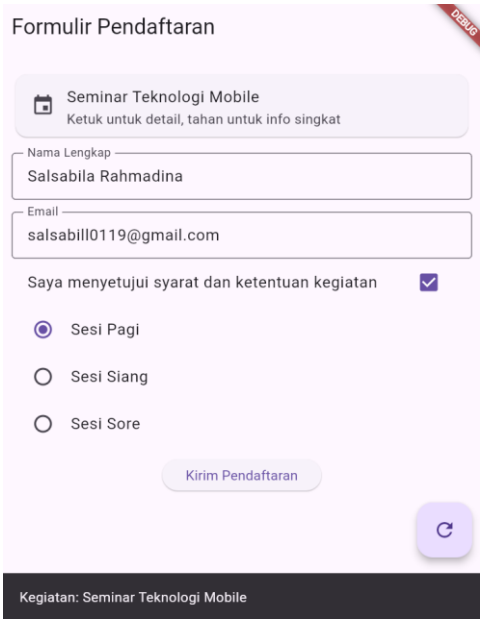


## 12. Menggunakan GestureDetector untuk Interaksi Kustom

Pada tahap ini, ditambahkan GestureDetector pada kartu informasi kegiatan untuk menangani dua jenis interaksi, yaitu sentuhan tunggal dan sentuhan panjang. Sentuhan tunggal digunakan untuk membuka halaman detail kegiatan, sedangkan sentuhan panjang menampilkan informasi singkat melalui SnackBar.

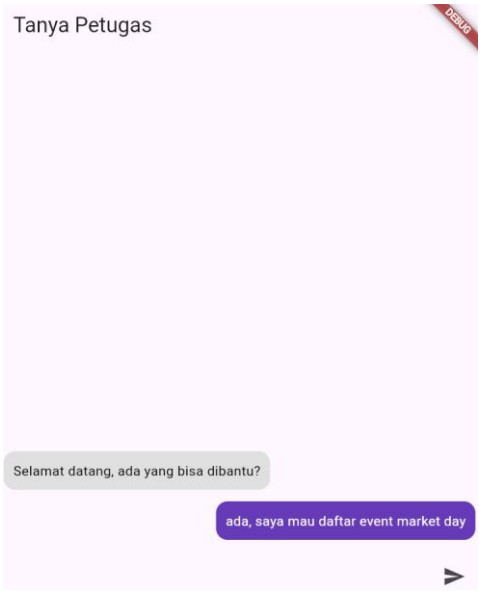



## 13. Mengintegrasikan Pustaka Chat Sederhana

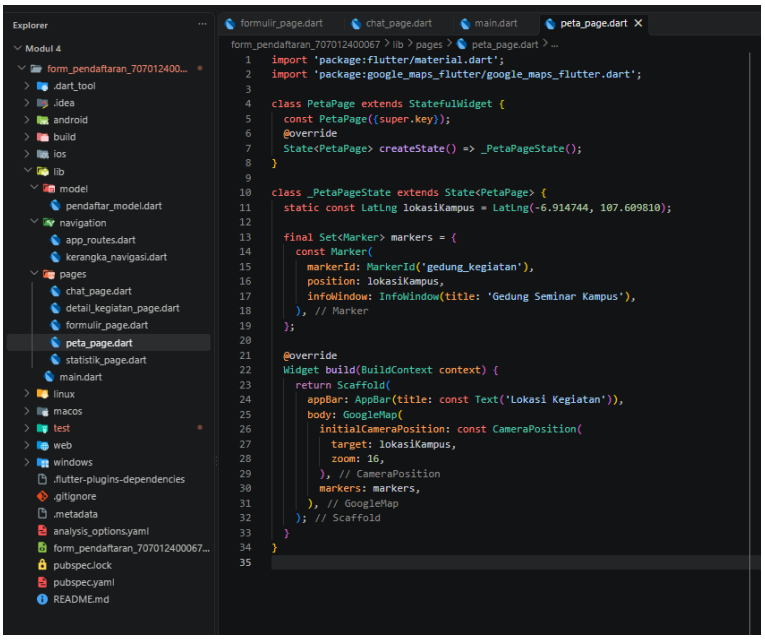
Pada tahap ini, dibuat halaman ChatPage menggunakan StatefulWidget untuk menampilkan percakapan sederhana antara peserta dan panitia. Pesan disimpan dalam state lokal dan ditampilkan menggunakan ListView.builder. Pengguna dapat mengetik pesan melalui TextField dan mengirimkannya menggunakan IconButton. Metode setState() digunakan untuk memperbarui daftar pesan setelah pesan dikirim.



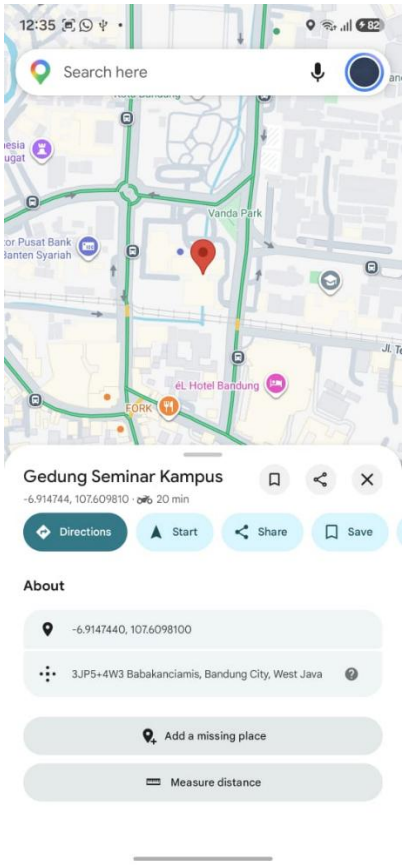



## 14. Mengintegrasikan Google Maps

Pada tahap ini, dibuat halaman PetaPage menggunakan pustaka google_maps_flutter untuk menampilkan lokasi kegiatan pada peta. Posisi awal kamera ditentukan menggunakan koordinat lintang dan bujur, sedangkan Marker digunakan untuk menandai lokasi Gedung Seminar Kampus. Informasi nama lokasi dapat ditampilkan ketika marker ditekan.



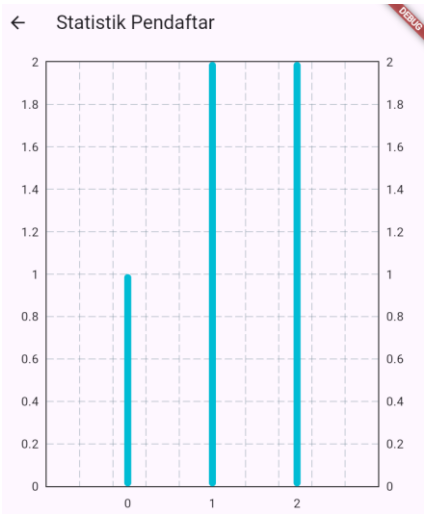



## 15. Mengintegrasikan Pustaka *Chart*

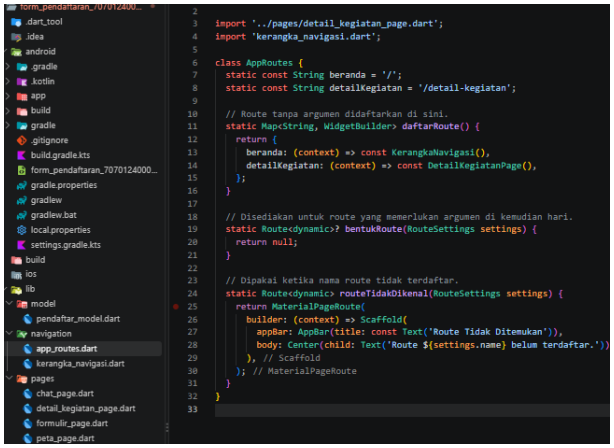
Pada tahap ini, dibuat halaman StatistikPage menggunakan pustaka fl_chart untuk menampilkan jumlah pendaftar dalam bentuk grafik batang (BarChart). Data jumlah pendaftar dihitung berdasarkan sesi Pagi, Siang, dan Sore melalui PendaftarModel. Dengan menggunakan context.watch(), grafik akan diperbarui secara otomatis ketika terdapat pendaftar baru yang ditambahkan.

```
PS D:\Kuliah\Semester 5\Pemprograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_707012400067> flutter pub add fl_chart
Resolving dependencies...
Downloading packages... (7.8s)
  cupertino_icons 1.0.9 (2.0.0 available)
+ equatable 2.1.0 (3.0.0 available)
+ fl_chart 1.2.0
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.15 available)
  vector_math 2.4.0 (2.4.3 available)
  vm_service 15.3.0 (15.4.0 available)
Changed 2 dependencies!
7 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
PS D:\Kuliah\Semester 5\Pemprograman Perangkat Bergerak Lanjut\Praktikum\Pekan 4\Modul 4\form_pendaftaran_707012400067>
```




## 16. Mengintegrasikan Pustaka Chart

Pada tahap ini, dibuat halaman StatistikPage menggunakan pustaka fl_chart untuk menampilkan jumlah pendaftar pada setiap sesi kegiatan dalam bentuk diagram batang. Data jumlah pendaftar sesi pagi, siang, dan sore diperoleh dari PendaftarModel melalui context.watch(). Dengan demikian, diagram akan diperbarui secara otomatis ketika terdapat pendaftar baru, sehingga petugas dapat melihat jumlah pendaftar pada setiap sesi dengan lebih mudah.



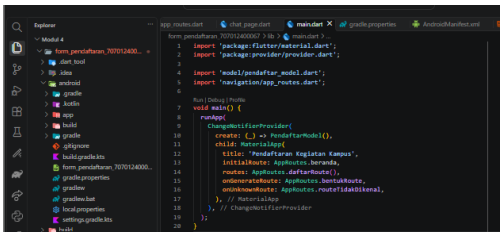
```
import '../pages/detail_kegiatan_page.dart';
import 'kerangka_navigasi.dart';

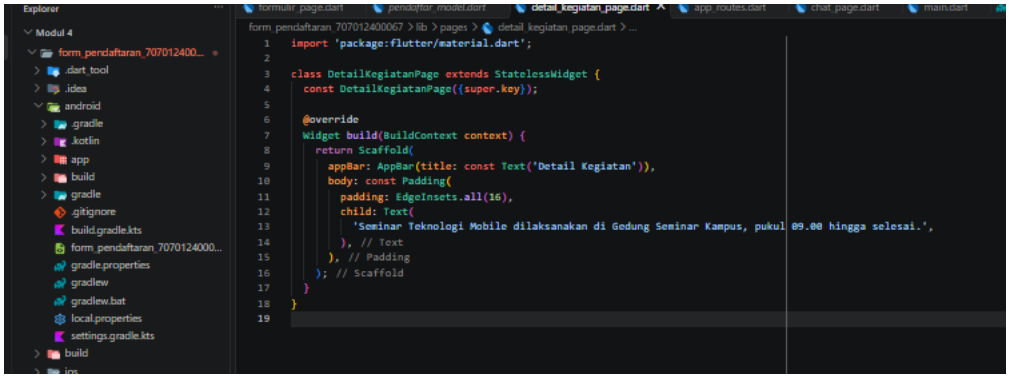
class AppRoutes {
  static const String beranda = '/';
  static const String detailKegiatan = '/detail-kegiatan';

  // Route tanpa argumen didaftarkan di sini.
  static Map<String, WidgetBuilder> daftarRoute() {
    return {
      beranda: (context) => const KerangkaNavigasi(),
      detailKegiatan: (context) => const DetailKegiatanPage(),
    };
  }

  // Disediakan untuk route yang memerlukan argumen di kemudian hari.
  static Route<dynamic>? bentukRoute(RouteSettings settings) {
    return null;
  }

  // Dipakai ketika nama route tidak terdaftar.
  static Route<dynamic> routeTidakDikenal(RouteSettings settings) {
    return MaterialPageRoute(
      builder: (context) => Scaffold(
        appBar: AppBar(title: const Text('Route Tidak Ditemukan')),
        body: Center(child: Text('Route ${settings.name} belum terdaftar.')),
      ), // Scaffold
    ); // MaterialPageRoute
  }
}
```

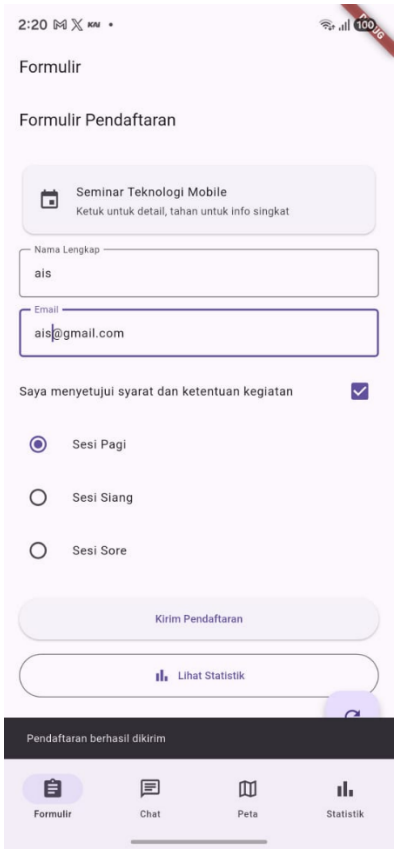


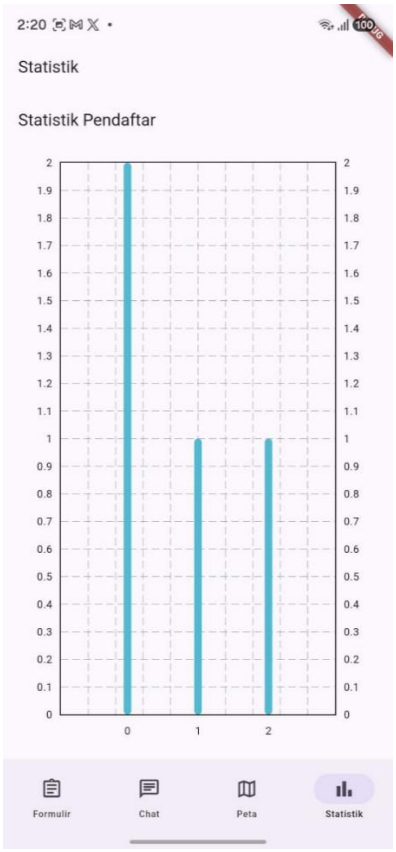


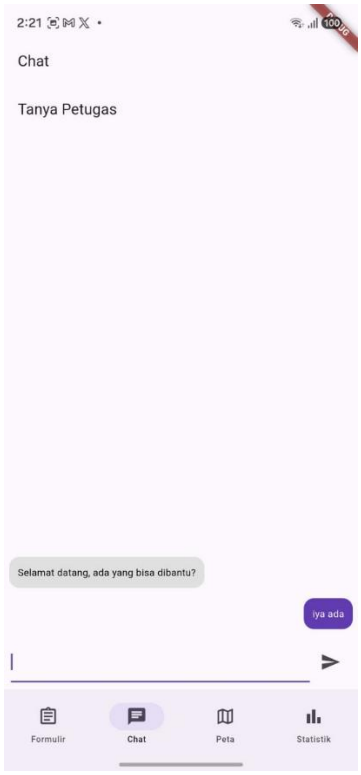
```
import 'package:flutter/material.dart';

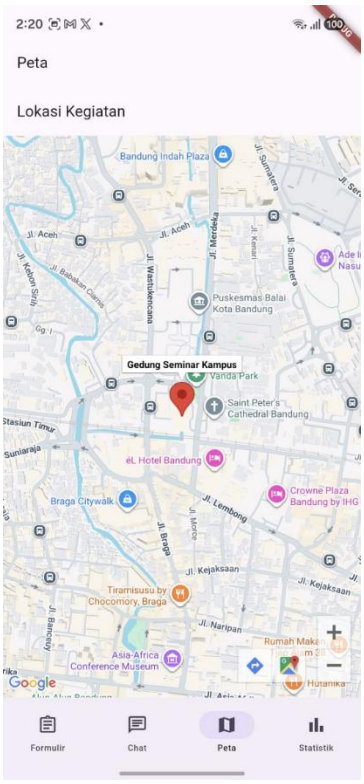
class DetailKegiatanPage extends StatelessWidget {
  const DetailKegiatanPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Detail Kegiatan')),
      body: const Padding(
        padding: EdgeInsets.all(16),
        child: Text(
          'Seminar Teknologi Mobile dilaksanakan di Gedung Seminar Kampus, pukul 09.00 hingga selesai.',
        ), // Text
      ), // Padding
    ); // Scaffold
  }
}
```


2:20
Formulir
Formulir Pendaftaran
Seminar Teknologi Mobile
Ketuk untuk detail, tahan untuk info singkat
Nama Lengkap
ais
Email
ais@gmail.com
Saya menyetujui syarat dan ketentuan kegiatan
Sesi Pagi
Sesi Siang
Sesi Sore
Kirim Pendaftaran
Lihat Statistik
Pendaftaran berhasil dikirim
Formulir
Chat
Peta
Statistik


2:20
Statistik
Statistik Pendaftar
Formulir
Chat
Peta
Statistik


2:21
Chat
Tanya Petugas
Selamat datang, ada yang bisa dibantu?
iya ada
Formulir
Chat
Peta
Statistik


2:20
Peta
Lokasi Kegiatan
Gedung Seminar Kampus
Bandung Indah Plaza
Puskesmas Balai Kota Bandung
Saint Peter's Cathedral Bandung
Braga Citywalk
Crowne Plaza Bandung by IHG
Asia-Africa Conference Museum
Formulir
Chat
Peta
Statistik

# Tugas Praktikum
# Aplikasi Layanan Warga Smart City “Nusantara Cerdas Mobile”

## 1. Tujuan Praktikum

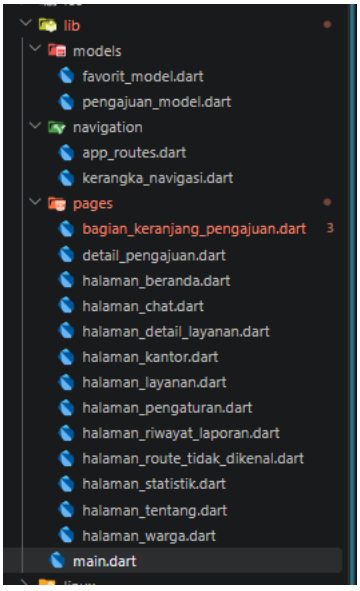
Praktikum ini bertujuan untuk menerapkan konsep form, interaksi pengguna, pengelolaan state, dan integrasi pustaka pada aplikasi Flutter melalui pengembangan aplikasi layanan warga “Nusantara Cerdas Mobile”. Praktikum ini mencakup pembuatan formulir pengajuan menggunakan TextField, Checkbox, dan RadioButton, penerapan validasi input, serta pengelolaan data pengajuan menggunakan ChangeNotifier dan Provider. Selain itu, praktikum ini bertujuan untuk menerapkan GestureDetector pada riwayat pengajuan, membangun halaman Tanya Petugas, menampilkan lokasi kantor menggunakan Google Maps, dan menyajikan statistik layanan dalam bentuk diagram batang.

## 2. Buat Projek Flutter

Proyek yang digunakan dalam praktikum ini adalah **Nusantara Cerdas Mobile**. Pengembangan dilakukan menggunakan Flutter melalui Visual Studio Code dengan memanfaatkan proyek yang telah tersedia sebelumnya. Proyek tersebut disesuaikan untuk memenuhi kebutuhan Modul 4, yaitu penambahan formulir pengajuan, interaksi pengguna, pengelolaan data, halaman komunikasi dengan petugas, peta lokasi kantor, dan statistik layanan.

## 3. Menyiapkan Struktur Berkas Proyek

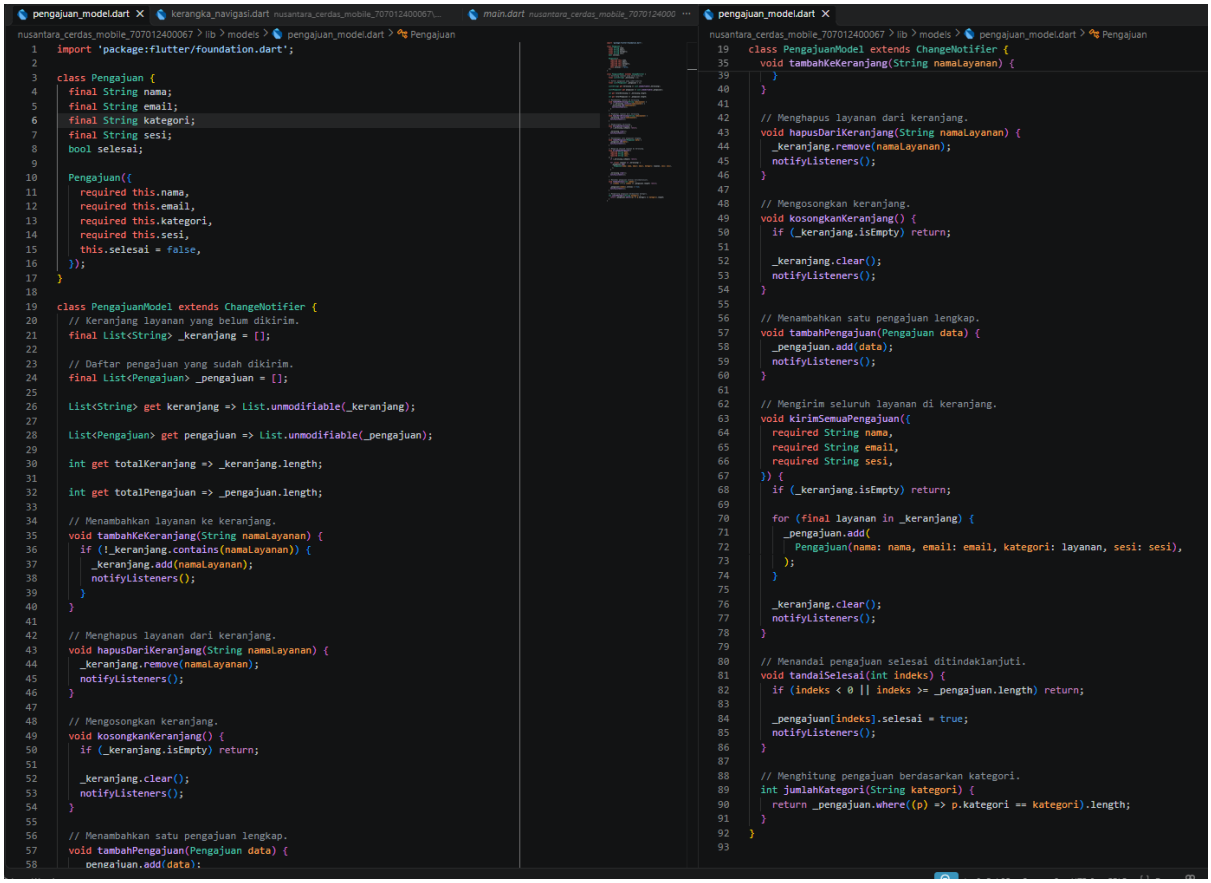
Struktur folder proyek disusun untuk memisahkan kode berdasarkan fungsinya agar lebih terorganisasi dan mudah dikembangkan. Folder lib digunakan untuk menyimpan kode utama aplikasi, yang terdiri atas folder model, navigation, dan pages, serta berkas main.dart.




## 4. Menyiapkan Model Data Pengajuan

Model data pengajuan dibuat menggunakan class PengajuanModel yang meng-extend ChangeNotifier. Model ini digunakan untuk menyimpan data pengajuan warga dan menyediakan informasi yang dapat digunakan oleh beberapa halaman aplikasi.

Model tersebut juga dilengkapi mekanisme pemberitahuan perubahan data melalui notifyListeners(), sehingga halaman yang menggunakan Provider dapat memperbarui tampilannya ketika data pengajuan berubah.



```dart
import 'package:flutter/foundation.dart';

class Pengajuan {
  final String nama;
  final String email;
  final String kategori;
  final String sesi;
  bool selesai;

  Pengajuan({
    required this.nama,
    required this.email,
    required this.kategori,
    required this.sesi,
    this.selesai = false,
  });
}

class PengajuanModel extends ChangeNotifier {
  // Keranjang layanan yang belum dikirim.
  final List<String> _keranjang = [];

  // Daftar pengajuan yang sudah dikirim.
  final List<Pengajuan> _pengajuan = [];

  List<String> get keranjang => List.unmodifiable(_keranjang);

  List<Pengajuan> get pengajuan => List.unmodifiable(_pengajuan);

  int get totalKeranjang => _keranjang.length;

  int get totalPengajuan => _pengajuan.length;

  // Menambahkan layanan ke keranjang.
  void tambahKeKeranjang(String namaLayanan) {
    if (!_keranjang.contains(namaLayanan)) {
      _keranjang.add(namaLayanan);
      notifyListeners();
    }
  }

  // Menghapus layanan dari keranjang.
  void hapusDariKeranjang(String namaLayanan) {
    _keranjang.remove(namaLayanan);
    notifyListeners();
  }

  // Mengosongkan keranjang.
  void kosongkanKeranjang() {
    if (_keranjang.isEmpty) return;

    _keranjang.clear();
    notifyListeners();
  }

  // Menambahkan satu pengajuan lengkap.
  void tambahPengajuan(Pengajuan data) {
    _pengajuan.add(data);
    notifyListeners();
  }

  // Mengirim seluruh layanan di keranjang.
  void kirimSemuaPengajuan({
    required String nama,
    required String email,
    required String sesi,
  }) {
    if (_keranjang.isEmpty) return;

    for (final layanan in _keranjang) {
      _pengajuan.add(
        Pengajuan(nama: nama, email: email, kategori: layanan, sesi: sesi),
      );
    }

    _keranjang.clear();
    notifyListeners();
  }

  // Menandai pengajuan selesai ditindaklanjuti.
  void tandaiSelesai(int indeks) {
    if (indeks < 0 || indeks >= _pengajuan.length) return;

    _pengajuan[indeks].selesai = true;
    notifyListeners();
  }

  // Menghitung pengajuan berdasarkan kategori.
  int jumlahKategori(String kategori) {
    return _pengajuan.where((p) => p.kategori == kategori).length;
  }
}
```

## 5. Mengatur Navigasi dan Routing Aplikasi

Navigasi aplikasi diatur menggunakan berkas app_routes.dart dan kerangka_navigasi.dart. Berkas app_routes.dart digunakan untuk mendaftarkan nama route dan menentukan halaman tujuan, sedangkan kerangka_navigasi.dart digunakan untuk menampilkan kerangka navigasi utama aplikasi.



```dart
import 'package:flutter/material.dart';

import '../pages/halaman_detail_layanan.dart';
import '../pages/halaman_riwayat_laporan.dart';
import '../pages/halaman_pengaturan.dart';
import '../pages/halaman_tentang.dart';

class AppRoutes {
  static const String detail = '/detail';
  static const String riwayat = '/riwayat';
  static const String pengaturan = '/pengaturan';
  static const String tentang = '/tentang';

  static Route<dynamic>? generateRoute(RouteSettings settings) {
    switch (settings.name) {
      case detail:
        final layanan = settings.arguments as Map<String, dynamic>;

        return MaterialPageRoute(
          builder: (_) => HalamanDetailLayanan(layanan: layanan),
        ); // MaterialPageRoute

      case riwayat:
        return MaterialPageRoute(
          builder: (_) => const HalamanRiwayatLaporan(),
        ); // MaterialPageRoute

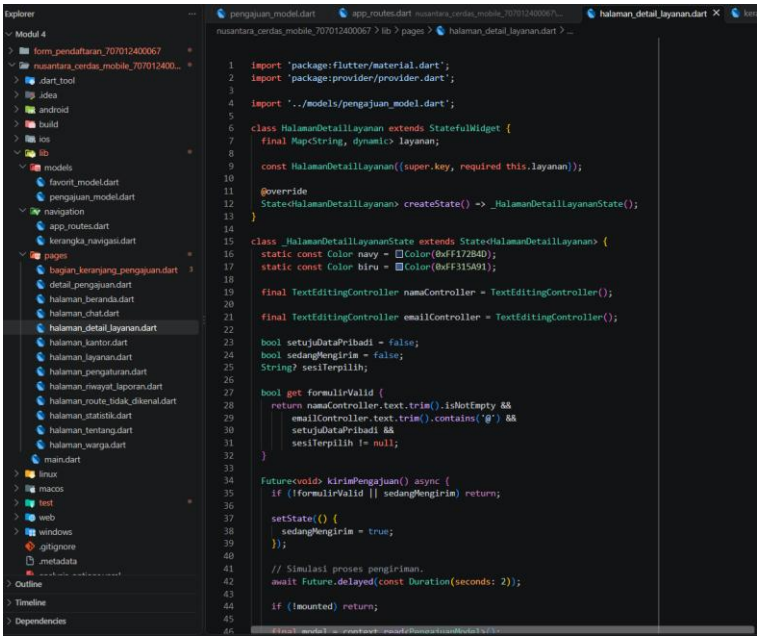
      case pengaturan:
        return MaterialPageRoute(
          builder: (_) => const HalamanPengaturan(),
        ); // MaterialPageRoute

      case tentang:
        return MaterialPageRoute(
          builder: (_) => const HalamanTentang(),
        ); // MaterialPageRoute

      default:
        return null;
    }
  }
}
```

### 6. Implementasi Formulir Pengajuan

Formulir pengajuan dirancang pada halaman rincian layanan dengan menggunakan TextField untuk memasukkan nama lengkap dan alamat email. Checkbox digunakan sebagai persetujuan penggunaan data pribadi, sedangkan RadioButton digunakan untuk memilih sesi kunjungan, yaitu pagi, siang, atau sore.



```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/pengajuan_model.dart';

class HalamanDetailLayanan extends StatefulWidget {
  final Map<String, dynamic> layanan;

  const HalamanDetailLayanan({super.key, required this.layanan});

  @override
  State<HalamanDetailLayanan> createState() => _HalamanDetailLayananState();
}

class _HalamanDetailLayananState extends State<HalamanDetailLayanan> {
  static const Color navy = Color(0xFF172B4D);
  static const Color biru = Color(0xFF315A91);

  final TextEditingController namaController = TextEditingController();

  final TextEditingController emailController = TextEditingController();

  bool setujuDataPribadi = false;
  bool sedangMengirim = false;
  String? sesiTerpilih;

  bool get formulirValid {
    return namaController.text.trim().isNotEmpty &&
        emailController.text.trim().contains('@') &&
        setujuDataPribadi &&
        sesiTerpilih != null;
  }

  Future<void> kirimPengajuan() async {
    if (!formulirValid || sedangMengirim) return;

    setState(() {
      sedangMengirim = true;
    });

    // Simulasi proses pengiriman.
    await Future.delayed(const Duration(seconds: 2));

    if (!mounted) return;
```

### 7. Implementasi Halaman Tanya Petugas

Halaman Tanya Petugas dirancang sebagai tampilan percakapan sederhana antara warga dan petugas. Daftar pesan ditampilkan menggunakan ListView.builder, sedangkan gelembung pesan dibedakan berdasarkan pengirimnya. Halaman ini juga dilengkapi kolom input dan tombol kirim untuk menambahkan pesan.



```dart
import 'package:flutter/material.dart';

class HalamanChat extends StatefulWidget {
  const HalamanChat({super.key});

  @override
  State<HalamanChat> createState() => _HalamanChatState();
}

class _HalamanChatState extends State<HalamanChat> {
  static const Color navy = Color(0xFF172B4D);
  static const Color biru = Color(0xFF315A91);
  static const Color background = Color(0xFFF4F6FA);

  final TextEditingController pesanController = TextEditingController();

  final ScrollController scrollController = ScrollController();

  final List<Map<String, dynamic>> daftarPesan = [
    {
      'isi': 'Halo! Selamat datang di layanan Tanya Petugas.',
      'dariWarga': false,
      'waktu': '09.00',
    },
    {
      'isi': 'Silakan sampaikan pertanyaan atau kendala Anda.',
      'dariWarga': false,
      'waktu': '09.00',
    },
  ];

  void kirimPesan() {
    final isiPesan = pesanController.text.trim();

    if (isiPesan.isEmpty) return;

    setState(() {
      daftarPesan.add({
        'isi': isiPesan,
        'dariWarga': true,
        'waktu':
            '${TimeOfDay.now().hour.toString().padLeft(2, '0')}:'
            '${TimeOfDay.now().minute.toString().padLeft(2, '0')}',
      });

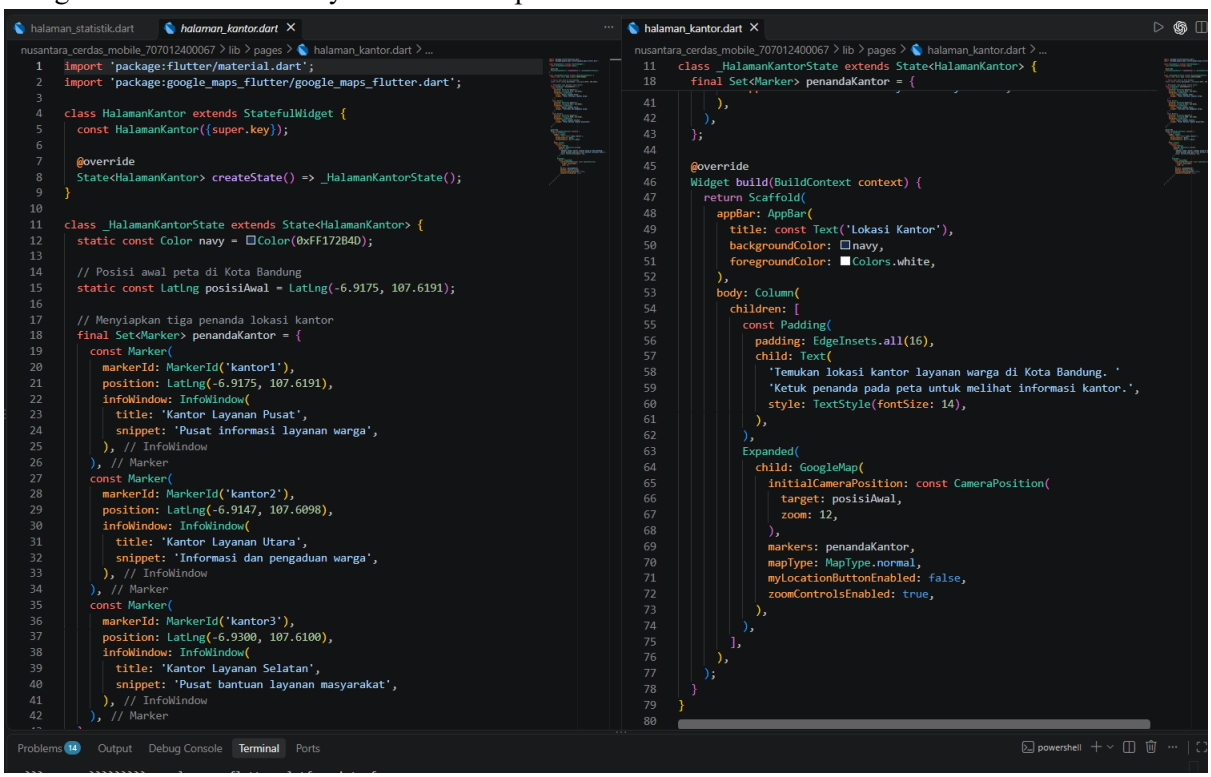
      pesanController.clear();
    });

    // Gulir ke pesan terbaru.
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (scrollController.hasClients) {
        scrollController.animateTo(
          scrollController.position.maxScrollExtent,
          duration: const Duration(milliseconds: 300),
          curve: Curves.easeOut,
        );
      }
    });
  }

  @override
  void dispose() {
    pesanController.dispose();
    scrollController.dispose();
    super.dispose();
  }
```

**9. Integrasi Google Maps pada Halaman Lokasi Kantor**

Halaman Lokasi Kantor menggunakan pustaka google_maps_flutter untuk menampilkan peta lokasi kantor layanan. Peta dilengkapi minimal tiga penanda lokasi kantor, dan setiap penanda menyediakan InfoWindow untuk menampilkan nama kantor layanan.Integrasi ini bertujuan membantu warga mengetahui lokasi kantor layanan melalui aplikasi.



```dart
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

class HalamanKantor extends StatefulWidget {
  const HalamanKantor({super.key});

  @override
  State<HalamanKantor> createState() => _HalamanKantorState();
}

class _HalamanKantorState extends State<HalamanKantor> {
  static const Color navy = Color(0xFF172B4D);

  // Posisi awal peta di Kota Bandung
  static const LatLng posisiAwal = LatLng(-6.9175, 107.6191);

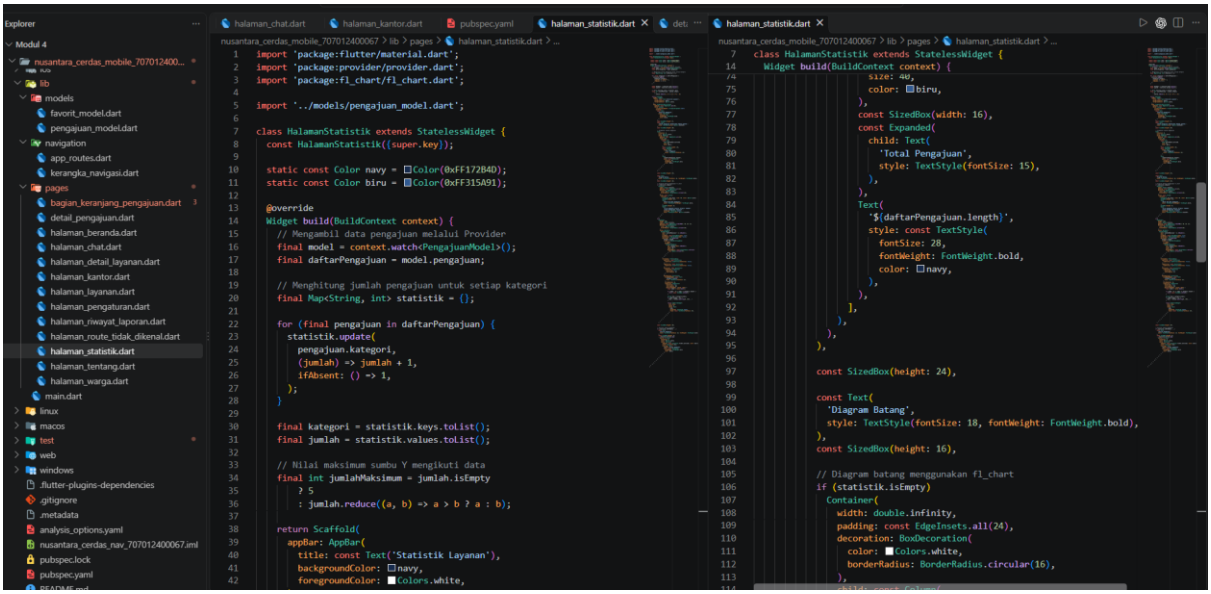
  // Menyiapkan tiga penanda lokasi kantor
  final Set<Marker> penandaKantor = {
    const Marker(
      markerId: MarkerId('kantor1'),
      position: LatLng(-6.9175, 107.6191),
      infoWindow: InfoWindow(
        title: 'Kantor Layanan Pusat',
        snippet: 'Pusat informasi layanan warga',
      ), // InfoWindow
    ), // Marker
    const Marker(
      markerId: MarkerId('kantor2'),
      position: LatLng(-6.9147, 107.6098),
      infoWindow: InfoWindow(
        title: 'Kantor Layanan Utara',
        snippet: 'Informasi dan pengaduan warga',
      ), // InfoWindow
    ), // Marker
    const Marker(
      markerId: MarkerId('kantor3'),
      position: LatLng(-6.9300, 107.6100),
      infoWindow: InfoWindow(
        title: 'Kantor Layanan Selatan',
        snippet: 'Pusat bantuan layanan masyarakat',
      ), // InfoWindow
    ), // Marker
```

```dart
class _HalamanKantorState extends State<HalamanKantor> {
  final Set<Marker> penandaKantor = {
    ),
  };

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Lokasi Kantor'),
        backgroundColor: navy,
        foregroundColor: Colors.white,
      ),
      body: Column(
        children: [
          const Padding(
            padding: EdgeInsets.all(16),
            child: Text(
              'Temukan lokasi kantor layanan warga di Kota Bandung. '
              'Ketuk penanda pada peta untuk melihat informasi kantor.',
              style: TextStyle(fontSize: 14),
            ),
          ),
          Expanded(
            child: GoogleMap(
              initialCameraPosition: const CameraPosition(
                target: posisiAwal,
                zoom: 12,
              ),
              markers: penandaKantor,
              mapType: MapType.normal,
              myLocationButtonEnabled: false,
              zoomControlsEnabled: true,
            ),
          ),
        ],
      ),
    );
  }
}
```

**10. Implementasi Statistik Layanan**

Halaman Statistik Layanan digunakan untuk menampilkan jumlah pengajuan berdasarkan bidang layanan. Data diambil dari PengajuanModel melalui Provider, kemudian disajikan dalam bentuk diagram batang menggunakan pustaka fl_chart.



```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:fl_chart/fl_chart.dart';

import '../models/pengajuan_model.dart';

class HalamanStatistik extends StatelessWidget {
  const HalamanStatistik({super.key});

  static const Color navy = Color(0xFF172B4D);
  static const Color biru = Color(0xFF315A91);

  @override
  Widget build(BuildContext context) {
    // Mengambil data pengajuan melalui Provider
    final model = context.watch<PengajuanModel>();
    final daftarPengajuan = model.pengajuan;

    // Menghitung jumlah pengajuan untuk setiap kategori
    final Map<String, int> statistik = {};

    for (final pengajuan in daftarPengajuan) {
      statistik.update(
        pengajuan.kategori,
        (jumlah) => jumlah + 1,
        ifAbsent: () => 1,
      );
    }

    final kategori = statistik.keys.toList();
    final jumlah = statistik.values.toList();

    // Nilai maksimum sumbu Y mengikuti data
    final int jumlahMaksimum = jumlah.isEmpty
        ? 5
        : jumlah.reduce((a, b) => a > b ? a : b);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Statistik Layanan'),
        backgroundColor: navy,
        foregroundColor: Colors.white,
```

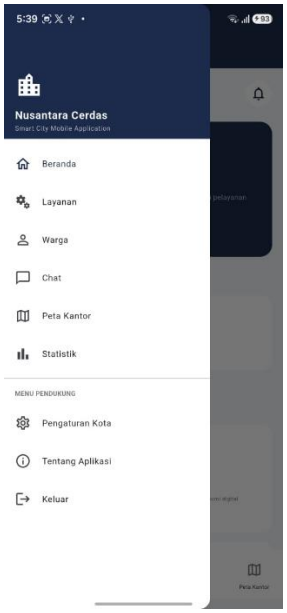
```dart
class HalamanStatistik extends StatelessWidget {
  Widget build(BuildContext context) {
                    size: 40,
                    color: biru,
                  ),
                  const SizedBox(width: 16),
                  const Expanded(
                    child: Text(
                      'Total Pengajuan',
                      style: TextStyle(fontSize: 15),
                    ),
                  ),
                  Text(
                    '${daftarPengajuan.length}',
                    style: const TextStyle(
                      fontSize: 28,
                      fontWeight: FontWeight.bold,
                      color: navy,
                    ),
                  ),
                ],
              ),
            ),
          ),

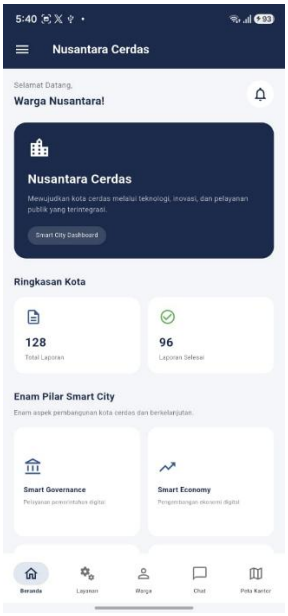
          const SizedBox(height: 24),

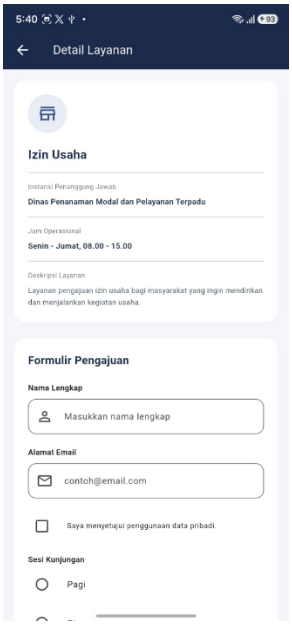
          const Text(
            'Diagram Batang',
            style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 16),

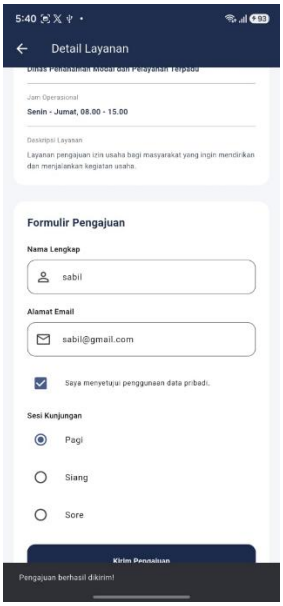
          // Diagram batang menggunakan fl_chart
          if (statistik.isEmpty)
            Container(
              width: double.infinity,
              padding: const EdgeInsets.all(24),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(16),
              ),
```

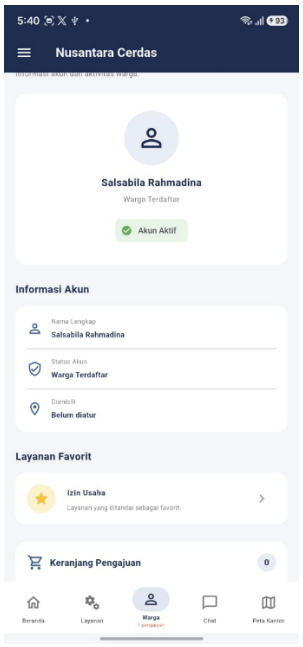
# 13. Hasil dan Dokumentasi Program

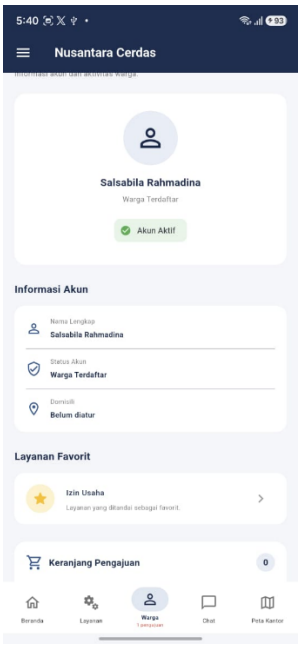


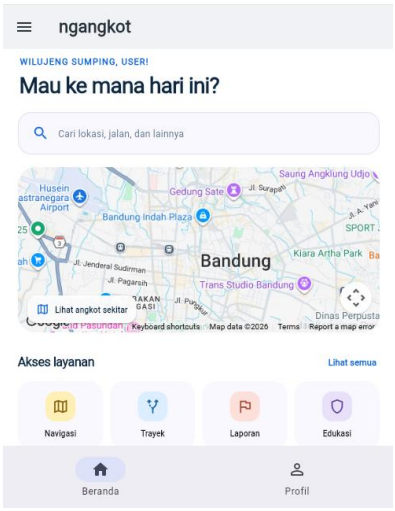



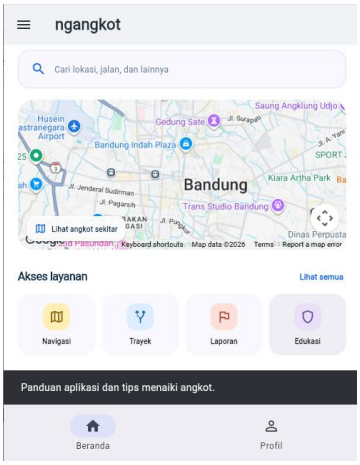



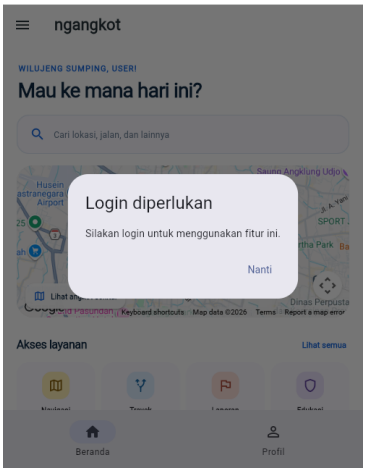


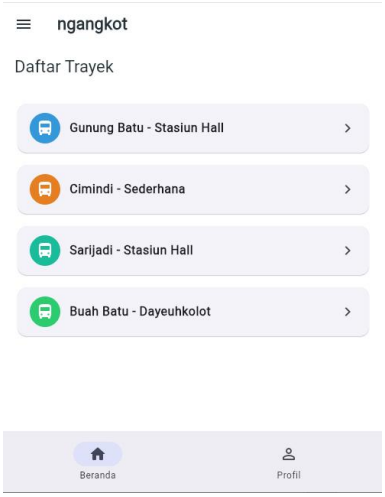

## SOAL 2. MERANCANG DAN MENGINTEGRASIKAN MATERI MODUL PADA RANCANGAN TUGAS AKHIR

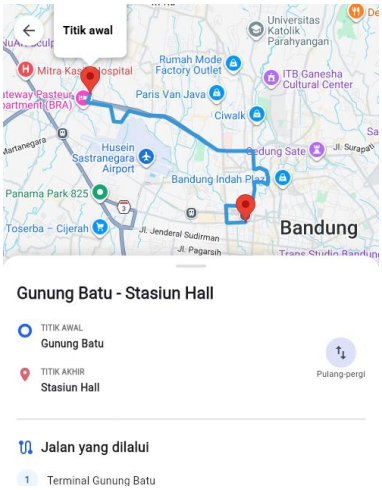
Aplikasi Ngangkot telah menerapkan beberapa fitur, yaitu halaman Beranda yang menampilkan peta dan akses layanan, popup login sebagai pemberitahuan ketika fitur tertentu memerlukan autentikasi, serta halaman Informasi Trayek yang menampilkan daftar trayek dan visualisasi rute angkot di Google Maps. Implementasi ini merupakan bagian dari pengembangan aplikasi mobile Ngangkot untuk memudahkan pengguna memperoleh informasi transportasi umum di Bandung.



# ANALISIS, KESIMPULAN, DAN LAMPIRAN

## 1. Analisis dan Pembahasan

Pada praktikum pertama, dilakukan pembelajaran dasar Flutter yang meliputi penggunaan StatelessWidget, StatefulWidget, layout, dan styling. StatelessWidget digunakan untuk menampilkan data yang tidak berubah, sedangkan StatefulWidget digunakan untuk tampilan yang dapat berubah melalui interaksi pengguna. Penggunaan setState() membantu memperbarui tampilan ketika terjadi perubahan data.

Pada praktikum Nusantara Cerdas Mobile, materi form, input, interaksi, dan pengelolaan state diterapkan dalam aplikasi layanan warga. Formulir menggunakan TextField, Checkbox, dan RadioButton untuk menerima input pengguna. Selain itu, aplikasi dikembangkan dengan halaman Tanya Petugas, Google Maps untuk menampilkan lokasi kantor, serta diagram batang untuk menyajikan statistik layanan. Provider dan ChangeNotifier digunakan untuk mengelola data agar dapat dimanfaatkan oleh beberapa halaman aplikasi.

Pada pengembangan tugas akhir Ngangkot, materi Flutter diterapkan pada aplikasi mobile informasi angkutan kota di Bandung. Halaman Beranda menampilkan peta, kolom pencarian, dan akses layanan, sedangkan popup login digunakan sebagai pemberitahuan ketika fitur tertentu memerlukan autentikasi. Fitur Informasi Trayek berhasil menampilkan daftar trayek, visualisasi rute pada Google Maps, titik awal dan akhir, serta informasi jalan yang dilalui. Pengambilan data trayek dilakukan melalui Dio Client yang terhubung dengan API Laravel. Namun, API masih menggunakan localhost sehingga backend perlu dijalankan agar data dapat ditampilkan.

## 2. Kesimpulan

Berdasarkan ketiga praktikum yang telah dilakukan, pemahaman mengenai pengembangan aplikasi menggunakan Flutter semakin berkembang, mulai dari penggunaan widget, penyusunan layout, styling, dan pengelolaan interaksi hingga integrasi pustaka dan API. Praktikum Nusantara Cerdas Mobile memberikan pengalaman dalam mengembangkan aplikasi layanan warga dengan form, pengelolaan state, peta, dan statistik. Sementara itu, pengembangan tugas akhir Ngangkot menerapkan materi Flutter untuk menampilkan informasi trayek dan visualisasi rute angkutan kota melalui integrasi API Laravel. Dengan demikian, materi praktikum dapat diterapkan secara bertahap dalam pengembangan aplikasi mobile yang lebih terstruktur dan sesuai kebutuhan pengguna.

## 3. Lampiran Link Repositori GitHub

Kode program lengkap dari praktikum disimpan pada repositori GitHub dan dapat diakses oleh dosen pengampu melalui tautan berikut:

**Link GitHub: https://github.com/salsabilrdn/Praktikum_PPBL/tree/main/Modul_4**

**Link GitHub Tubes (Ngangkot) : https://github.com/belvaedria/ngangkot-app**